Rapport de POO en Java

Gestion des marchandises et des etudiants

Hiba EL BOUHADDIOUI

Rédigé le 16 octobre 2026 à 11:21

Exercice Complémentaire : Gestion des Marchandises, Personnes et Utilisateurs

Description et Objectif

Cet exercice complet met en œuvre les principes de la programmation orientée objet (POO) en Java, incluant l'encapsulation, les constructeurs surchargés, le mécanisme d'héritage (la classe `Utilisateur` étendant la classe `Personne`), l'utilisation de collections (`List` et `ArrayList`), ainsi qu'un système d'authentification console interactif via la classe `Scanner`.

Code Source Java

```java
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Scanner;

public class ex1 {

    public static void main(String[] args) {

        // ==================== MARCHANDISE ====================

        Marchandise m1 = new Marchandise("M001", "Ordinateur");
        Marchandise m2 = new Marchandise("M002");

        System.out.println(m1);
        System.out.println(m2);

        // ==================== UTILISATEURS ====================

        List<Utilisateur> utilisateurs = new ArrayList<>();

        utilisateurs.add(new Utilisateur(
                1, "hiba", "hiba", 6000, "admin", "1234"));

        utilisateurs.add(new Utilisateur(
                2, "sara", "sara", 5000, "sara", "abcd"));

        // ==================== AUTHENTIFICATION ====================

        Scanner scanner = new Scanner(System.in);

        System.out.print("Login : ");
        String login = scanner.nextLine();

```

```
        System.out.print("Password : ");
        String password = scanner.nextLine();

        Utilisateur trouve = null;

        for (Utilisateur utilisateur : utilisateurs) {
            if (utilisateur.getLogin().equals(login)
                    && utilisateur.getPassword().equals(password)) {
                trouve = utilisateur;
                break;
            }
        }

        if (trouve != null) {
            System.out.println("Authentification reussie");
            trouve.afficheUtilisateur();
        } else {
            System.out.println("Echec d'Authentification");
        }

        scanner.close();
    }
}

// ============================================================
// CLASSE MARCHANDISE
// ============================================================

class Marchandise {

    private String code;
    private String description;
    private final Date dateAchat;

    public Marchandise() {
        this("", "");
    }

    public Marchandise(String code, String description) {
        this.code = code;
        this.description = description;
        this.dateAchat = new Date();
    }

    public Marchandise(String code) {
        this(code, "");
    }

    public Marchandise(Marchandise autre) {
        this(autre.code, autre.description);
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public String getDescription() {
```

```
96          return description;
97      }
98
99      public void setDescription(String description) {
100         this.description = description;
101     }
102
103     public Date getDateAchat() {
104         return new Date(dateAchat.getTime());
105     }
106
107     @Override
108     public String toString() {
109         return "Le produit \u00ab " + code + " \u00bb - \u00ab " +
                description
110                 + " \u00bb a ete achete le \u00ab " + dateAchat + "
                    \u00bb.";
111     }
112 }
113
114 // ============================================================
115 // CLASSE PERSONNE
116 // ============================================================
117
118 class Personne {
119
120     private int id;
121     private String nom;
122     private String prenom;
123     private double salaire;
124
125     public Personne() {
126     }
127
128     public Personne(int id, String nom, String prenom, double salaire) {
129         this.id = id;
130         this.nom = nom;
131         this.prenom = prenom;
132         this.salaire = salaire;
133     }
134
135     public Personne(Personne autre) {
136         this(autre.id, autre.nom, autre.prenom, autre.salaire);
137     }
138
139     public int getId() {
140         return id;
141     }
142
143     public void setId(int id) {
144         this.id = id;
145     }
146
147     public String getNom() {
148         return nom == null ? null : nom.toUpperCase();
149     }
150
151     public void setNom(String nom) {
152         this.nom = nom;
153     }
154
```

```
155     public String getPrenom() {
156         return prenom == null ? null : prenom.toUpperCase();
157     }
158 
159     public void setPrenom(String prenom) {
160         this.prenom = prenom;
161     }
162 
163     public double getSalaire() {
164         return salaire;
165     }
166 
167     public void setSalaire(double salaire) {
168         this.salaire = salaire;
169     }
170 
171     public void affichePersonne() {
172         System.out.println("ID : " + id
173                 + ", Nom : " + getNom()
174                 + ", Prenom : " + getPrenom()
175                 + ", Salaire : " + salaire);
176     }
177 }
178 
179 // ============================================================
180 // CLASSE UTILISATEUR
181 // ============================================================
182 
183 class Utilisateur extends Personne {
184 
185     private String login;
186     private String password;
187 
188     public Utilisateur() {
189     }
190 
191     public Utilisateur(int id, String nom, String prenom, double salaire,
        String login, String password) {
192         super(id, nom, prenom, salaire);
193         this.login = login;
194         this.password = password;
195     }
196 
197     public String getLogin() {
198         return login;
199     }
200 
201     public void setLogin(String login) {
202         this.login = login;
203     }
204 
205     public String getPassword() {
206         return password;
207     }
208 
209     public void setPassword(String password) {
210         this.password = password;
211     }
212 
213     public void afficheUtilisateur() {
214         super.affichePersonne();
```

```
215             System.out.println("Login : " + login + ", Password : " + password);
216         }
217 }
```

Code Source 1 – Code source de l'exercice complet (Classes Main, Marchandise, Personne et Utilisateur)

Resultat de l'execution

Apres la compilation du fichier `ex1.java` avec la commande `javac ex1.java`, le programme peut etre execute avec `java ex1`.

Cas 1 : Authentification reussie

Dans ce premier cas, l'utilisateur saisit un login et un mot de passe existant dans la liste des utilisateurs.

Code Source 2 – Execution avec des identifiants valides

```
C:\TPJava> javac ex1.java
C:\TPJava> java ex1

Le produit " M001 " - " Ordinateur " a ete achete le " Fri Oct 16 10:56:00
    2026 ".
Le produit " M002 " - " " a ete achete le " Fri Oct 16 10:56:00 2026 ".

Login : admin
Password : 1234

Authentification reussie
ID : 1, Nom : HIBA, Prenom : HIBA, Salaire : 6000.0
Login : admin, Password : 1234
```

Cas 2 : Echec d'authentification

Dans ce deuxieme cas, l'utilisateur saisit un mot de passe incorrect. Le programme ne trouve donc aucun utilisateur correspondant aux identifiants fournis.

Code Source 3 – Execution avec des identifiants invalides

```
C:\TPJava> java ex1

Le produit " M001 " - " Ordinateur " a ete achete le " Fri Oct 16 10:56:00
    2026 ".
Le produit " M002 " - " " a ete achete le " Fri Oct 16 10:56:00 2026 ".

Login : admin
Password : incorrect

Echec d'Authentification
```

Remarque : La date et l'heure affichees dependent du moment de l'execution du programme. Les sorties ci-dessus sont des exemples illustratifs.

Conclusion

Cet exercice a permis de mettre en pratique les principaux concepts de la programmation orientee objet en Java : l'encapsulation, la surcharge des constructeurs, l'heritage et la redefinition de la methode `toString()`.

Il a egalement permis de manipuler une collection d'objets `Utilisateur` et de realiser une authentification simple a partir des informations saisies au clavier.

Exercice 2 : Gestion des Etudiants

Description et Objectif

Cet exercice a pour objectif de mettre en pratique les concepts fondamentaux de la programmation orientee objet en Java, notamment l'encapsulation, l'heritage et la redefinition des methodes.

Il consiste a creer une classe generale `Etudiant`, puis deux classes derivees : `EtudiantFaculte` et `EtudiantENSA`.

Chaque classe derivee redefinit la methode `AfficheEtudiant()` afin de personnaliser l'affichage selon le type d'etablissement.

Code Source Java

```
1
2  // ============================================================
3  // CLASSE PRINCIPALE : EX2
4  // ============================================================
5
6  public class ex2 {
7
8      public static void main(String[] args) {
9
10         // Creation d'un objet de la classe Etudiant.
11         // Le constructeur initialise le CIN, le nom et l'age.
12         Etudiant e = new Etudiant("AB123456", "Alami", 20);
13
14         // Appel de la methode d'affichage de Etudiant.
15         e.AfficheEtudiant();
16         System.out.println();
17
18
19         // Creation d'un etudiant de la faculte.
20         // EtudiantFaculte herite de la classe Etudiant.
21         EtudiantFaculte ef = new EtudiantFaculte(
22                 "CD789012", "Bennani", 21);
23
24         // Appel de la methode redefinie dans EtudiantFaculte.
25         ef.AfficheEtudiant();
26         System.out.println();
27
28
29         // Creation d'un etudiant de l'ENSA.
30         // EtudiantENSA herite egalement de Etudiant.
31         EtudiantENSA ee = new EtudiantENSA(
32                 "EF345678", "Idrissi", 22);
33
34         // Appel de la methode redefinie dans EtudiantENSA.
35         ee.AfficheEtudiant();
36         System.out.println();
37
38
39         // ==================== TEST DES SETTERS ====================
40
41         // Modification du nom de l'etudiant ENSA.
42         ee.setNom("Idrissi Karim");
43
44         // Modification de l'age de l'etudiant ENSA.
45         ee.setAge(23);
46
47         // Verification des modifications par affichage.
```

```
48          ee.AfficheEtudiant();
49      }
50  }
51
52
53  // ============================================================
54  // CLASSE ETUDIANT
55  // ============================================================
56
57  // Classe mere representant un etudiant.
58  // Elle contient les attributs communs aux differents
59  // types d'etudiants.
60  class Etudiant {
61
62      // Attributs prives : principe de l'encapsulation.
63      // Ils ne sont pas directement accessibles
64      // depuis les classes derivees.
65      private String CIN;
66      private String Nom;
67      private int Age;
68
69
70      // ==================== CONSTRUCTEURS ====================
71
72      // Constructeur par defaut.
73      // Les attributs prennent leurs valeurs par defaut :
74      // null pour String et 0 pour int.
75      public Etudiant() {
76      }
77
78      // Constructeur d'initialisation.
79      // Il permet d'initialiser tous les attributs.
80      public Etudiant(String CIN, String Nom, int Age) {
81
82          // this designe l'objet courant.
83          // this.CIN est l'attribut de la classe,
84          // alors que CIN est le parametre du constructeur.
85          this.CIN = CIN;
86          this.Nom = Nom;
87          this.Age = Age;
88      }
89
90
91      // ==================== GETTERS / SETTERS ====================
92
93      // Getter permettant de lire le CIN.
94      public String getCIN() {
95          return CIN;
96      }
97
98      // Setter permettant de modifier le CIN.
99      public void setCIN(String CIN) {
100         this.CIN = CIN;
101     }
102
103     // Getter permettant de lire le nom.
104     public String getNom() {
105         return Nom;
106     }
107
108     // Setter permettant de modifier le nom.
```

```java
    public void setNom(String Nom) {
        this.Nom = Nom;
    }

    // Getter permettant de lire l'age.
    public int getAge() {
        return Age;
    }

    // Setter permettant de modifier l'age.
    public void setAge(int Age) {
        this.Age = Age;
    }


    // ==================== METHODE D'AFFICHAGE ====================

    // Affiche les informations generales d'un etudiant.
    // Cette methode sera redefinie dans les classes filles.
    public void AfficheEtudiant() {

        System.out.println(
            "CIN : " + CIN
            + "    Nom : " + Nom
            + "    Age : " + Age
        );
    }
}


// ============================================================
// CLASSE ETUDIANTFACULTE
// ============================================================

// extends indique que EtudiantFaculte herite de Etudiant.
// Cette classe beneficie des methodes publiques
// de sa classe mere.
class EtudiantFaculte extends Etudiant {

    // Constructeur avec trois parametres.
    public EtudiantFaculte(String CIN, String Nom, int Age) {

        // super(...) appelle le constructeur de la classe mere.
        // Il initialise les attributs CIN, Nom et Age.
        super(CIN, Nom, Age);
    }


    // ==================== REDEFINITION ====================

    // @Override indique que cette methode redefinit
    // une methode deja presente dans la classe mere.
    @Override
    public void AfficheEtudiant() {

        // Affichage specifique aux etudiants de la faculte.
        System.out.println("Etudiant a la faculte :");

        // Les attributs etant prives dans Etudiant,
        // on utilise les getters herites pour les consulter.
        System.out.println(
```

```
            "CIN : " + getCIN()
            + "   Nom : " + getNom()
            + "   Age : " + getAge()
        );
    }
}


// ============================================================
// CLASSE ETUDIANTENSA
// ============================================================

// EtudiantENSA herite aussi de la classe Etudiant.
// Il s'agit d'une autre classe fille de Etudiant.
class EtudiantENSA extends Etudiant {

    // Constructeur d'initialisation.
    public EtudiantENSA(String CIN, String Nom, int Age) {

        // Appel du constructeur de la classe mere.
        super(CIN, Nom, Age);
    }


    // ==================== REDEFINITION ====================

    // La methode AfficheEtudiant() est redefinie
    // pour adapter l'affichage aux etudiants de l'ENSA.
    @Override
    public void AfficheEtudiant() {

        System.out.println("Etudiant a l'ENSA :");

        // Recuperation des attributs par les getters herites.
        System.out.println(
            "CIN : " + getCIN()
            + "   Nom : " + getNom()
            + "   Age : " + getAge()
        );
    }
}
```

Code Source 4 – Code source complet de l'exercice 2

Resultat de l'execution

Le programme a ete compile et execute a partir du terminal avec les commandes suivantes :

```
javac ex2.java
java ex2
```

Test 1 : Affichage des trois types d'etudiants

Le premier test permet de verifier l'affichage des informations d'un etudiant ordinaire, d'un etudiant de la faculte et d'un etudiant de l'ENSA.

```
C:\TPJava> javac ex2.java
C:\TPJava> java ex2

CIN : AB123456   Nom : Alami   Age : 20
```

```
Etudiant a la faculte :
CIN : CD789012   Nom : Bennani   Age : 21

Etudiant a l'ENSA :
CIN : EF345678   Nom : Idrissi   Age : 22

Etudiant a l'ENSA :
CIN : EF345678   Nom : Idrissi Karim   Age : 23
```

Code Source 5 – Affichage des informations des etudiants

Test 2 : Modification des attributs

Ce test verifie le fonctionnement des setters `setNom()` et `setAge()`.

Avant la modification, l'etudiant ENSA possede les informations suivantes :

```
Etudiant a l'ENSA :
CIN : EF345678   Nom : Idrissi   Age : 22
```

Apres les instructions :

```
1 ee.setNom("Idrissi Karim");
2 ee.setAge(23);
3 ee.AfficheEtudiant();
```

Le resultat obtenu est :

```
Etudiant a l'ENSA :
CIN : EF345678   Nom : Idrissi Karim   Age : 23
```

Ces resultats confirment que les valeurs des attributs peuvent etre modifiees apres la creation de l'objet grace aux setters herites de la classe `Etudiant`.

Conclusion

Cet exercice a permis de comprendre le mecanisme d'heritage en Java en utilisant une classe mere `Etudiant` et deux classes derivees : `EtudiantFaculte` et `EtudiantENSA`.

La redefinition de la methode `AfficheEtudiant()` permet de personnaliser l'affichage selon le type d'etudiant, tout en conservant les attributs et les methodes communs dans la classe mere.

Les tests ont egalement permis de verifier le bon fonctionnement des constructeurs, des accesseurs et des modificateurs.